

Assignment 5: Cron Job Scheduling for Automated Backups

Documentation & Execution Proof

1. Scenario

A critical application generates daily reports in /var/app/reports. The goal is to set up an automated system that:

- Archives these reports daily at 01:00 AM into a compressed (.tar.gz) file
- Moves archives older than 3 days from the daily folder to a long-term storage folder
- Deletes archives from long-term storage once they are older than 7 days

2. Preparing the Directory Structure

Before writing the script, the required directories were created on the server:

- /var/app/reports – source folder where daily reports are generated
- /var/backups/daily – destination for the daily compressed backups
- /var/backups/longterm – long-term storage for older backups

Command used:

```
sudo mkdir -p /var/app/reports
sudo mkdir -p /var/backups/daily
sudo mkdir -p /var/backups/longterm
```

3. Creating the Backup Script

A shell script named daily_report_backup.sh was created at /usr/local/bin/ using the command:

```
sudo vi /usr/local/bin/daily_report_backup.sh
```

The script performs three main actions, in order:

- Creates a compressed archive of /var/app/reports with a timestamped filename (reports_YYYYMMDD.tar.gz) and saves it into /var/backups/daily
- Finds archives in /var/backups/daily older than 3 days (using find -mtime +3) and moves them into /var/backups/longterm
- Finds archives in /var/backups/longterm older than 7 days (using find -mtime +7) and permanently deletes them

Full script content (viewed with cat):

```
#!/bin/bash

# Directories
SOURCE="/var/app/reports"
DAILY="/var/backups/daily"
LONGTERM="/var/backups/longterm"

# Create timestamp
DATE=$(date +"%Y%m%d")

# Backup filename
BACKUP_FILE="$DAILY/reports_${DATE}.tar.gz"

# Create compressed backup
tar -czf "$BACKUP_FILE" -C /var/app reports

echo "Backup created: $BACKUP_FILE"

# Move daily archives older than 3 days to long-term storage
find "$DAILY" -type f -name "reports_*.tar.gz" -mtime +3 -exec mv {} "$LONGTERM/" \;

echo "Archives older than 3 days moved to long-term storage."

# Delete long-term archives older than 7 days
find "$LONGTERM" -type f -name "reports_*.tar.gz" -mtime +7 -delete

echo "Long-term archives older than 7 days deleted."

echo "Backup process completed successfully."
```

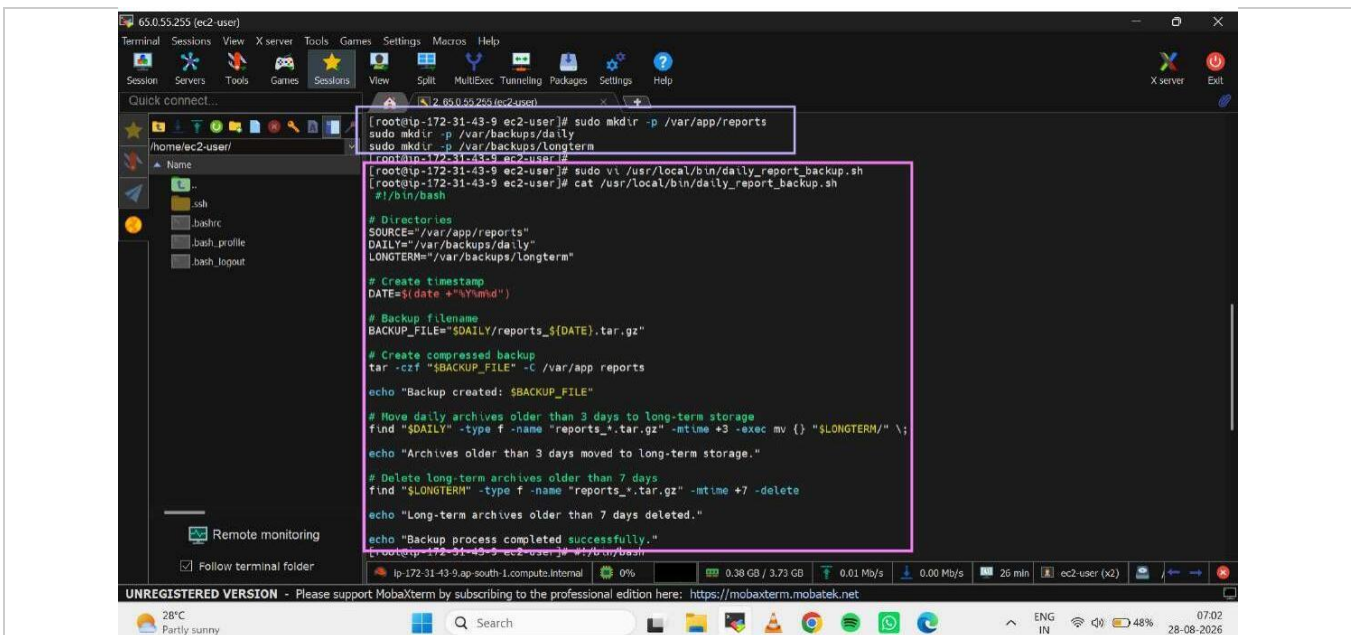


Fig 1: Directories created and daily_report_backup.sh script content.

4. Making the Script Executable

Execute permission was granted to the script so it can be run directly and by cron:

```
sudo chmod +x /usr/local/bin/daily_report_backup.sh
ls -l /usr/local/bin/daily_report_backup.sh
```

The output confirms the permission bits changed to -rwxr-xr-x, meaning the file is now executable.

5. Verification – Creating Dummy Data

5.1 Dummy report files

Sample folders and report files were created inside /var/app/reports to simulate real application data:

```
sudo mkdir -p /var/app/reports/2026
sudo mkdir -p /var/app/reports/2025

sudo touch /var/app/reports/today_report.txt
sudo touch /var/app/reports/2026/sales_report.txt
sudo touch /var/app/reports/2025/old_report.txt
```

ls -lR /var/app/reports confirms both sub-folders (2025, 2026) and all three dummy files exist before the script is run.

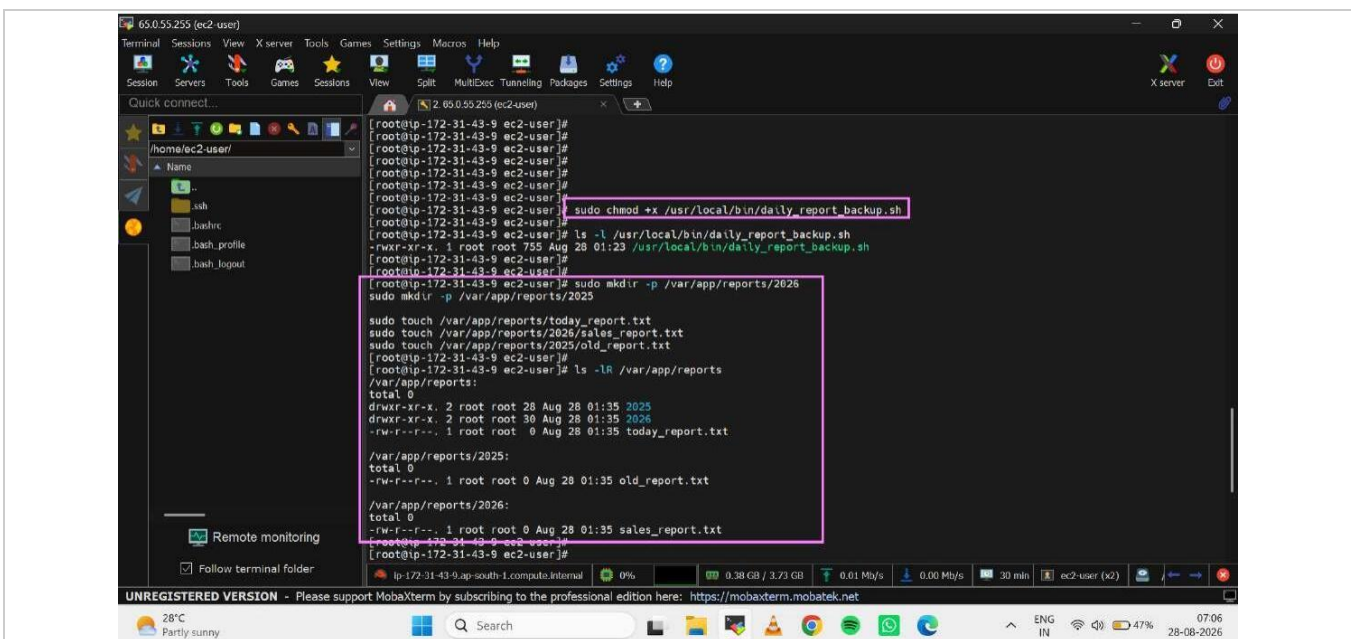


Fig 2: Script made executable (chmod +x) and dummy report files/folders created under /var/app/reports.

5.2 Dummy archives with backdated timestamps

To simulate the passage of time without waiting several days, dummy .tar.gz files were created in the daily and long-term folders and their modification times were backdated using touch -d:

```
sudo touch -d "4 days ago" /var/backups/daily/reports_older_4days.tar.gz
sudo touch -d "1 day ago" /var/backups/daily/reports_recent.tar.gz
sudo touch -d "8 days ago" /var/backups/longterm/reports_older_8days.tar.gz
sudo touch -d "2 days ago" /var/backups/longterm/reports_recent.tar.gz
```

This gave a realistic “before” state: one file older than 3 days sitting in /daily (should move), and one file older than 7 days sitting in /longterm (should be deleted), alongside two recent files that should be left untouched.

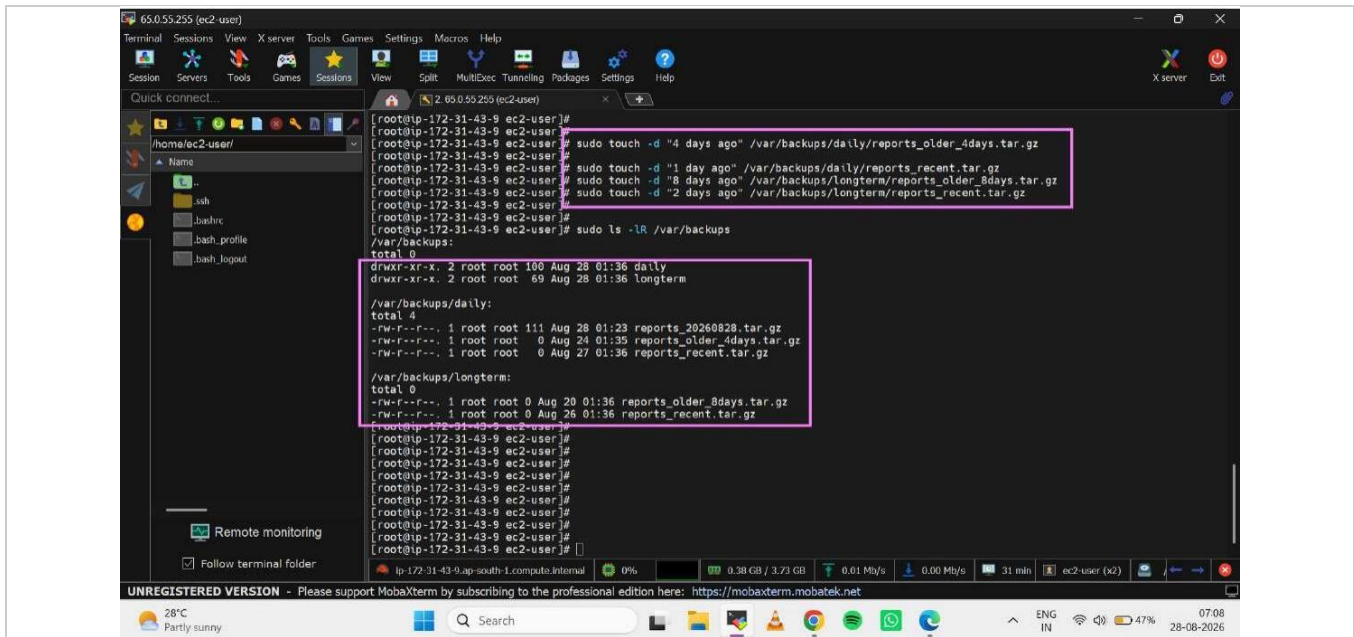


Fig 3: Backdated dummy archive files created, and the state of /var/backups (daily & longterm) before running the script.

6. Scheduling the Script with Cron

The cron service was verified and enabled to start automatically on boot, then the job was scheduled to run daily at 01:00 AM:

```

systemctl enable --now crond
systemctl status crond

sudo crontab -e

```

Crontab entry added:

```

0 1 * * * /usr/local/bin/daily_report_backup.sh >> /var/log/daily_report_backup.log 2>&1

```

This runs the script every day at 01:00 AM (minute 0, hour 1) and redirects both standard output and error output to a log file for auditing.

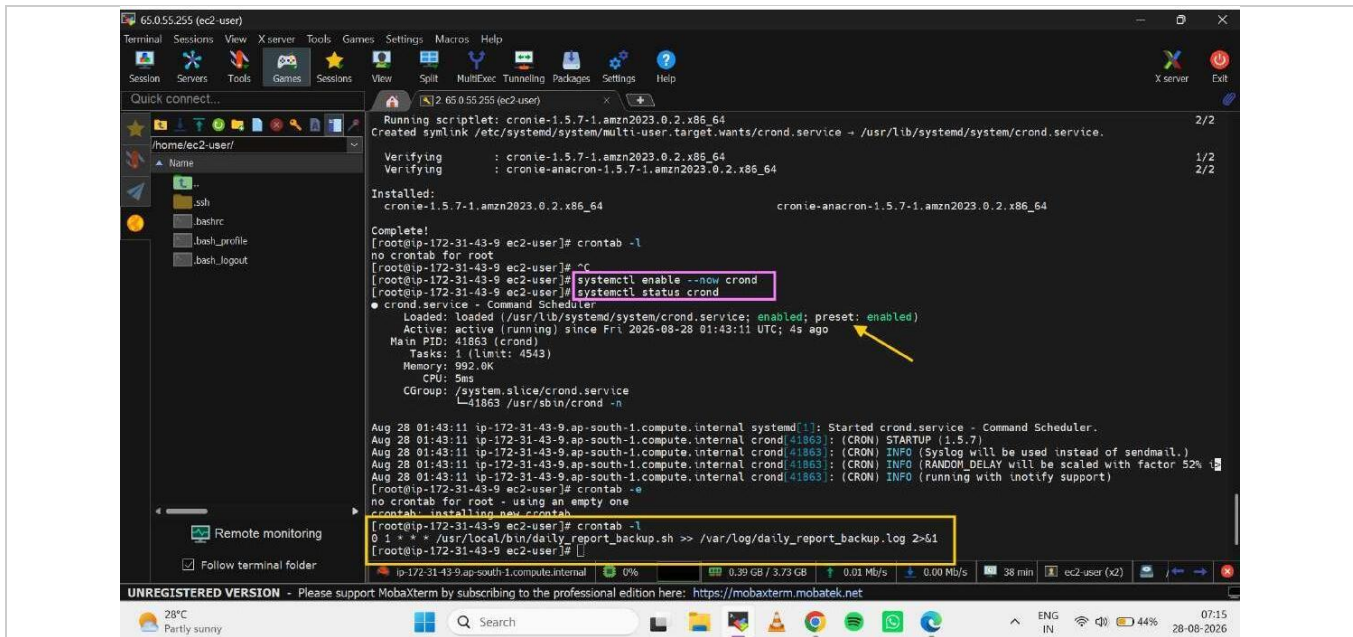


Fig 4: crond service enabled & active, and crontab -l showing the scheduled job.

7. Manual Run & Result Verification

The script was executed manually (simulating the scheduled run) and the archive produced was inspected with `tar -tzf` to confirm its contents:

```
sudo tar -tzf /var/backups/daily/reports_20260828.tar.gz
```

The listing shows the full folder structure was preserved inside the archive — `reports/`, `reports/2026/sales_report.txt`, `reports/2025/old_report.txt`, and `reports/today_report.txt` — confirming the backup was created correctly. The final `crontab -l` output re-confirms the job remains scheduled.

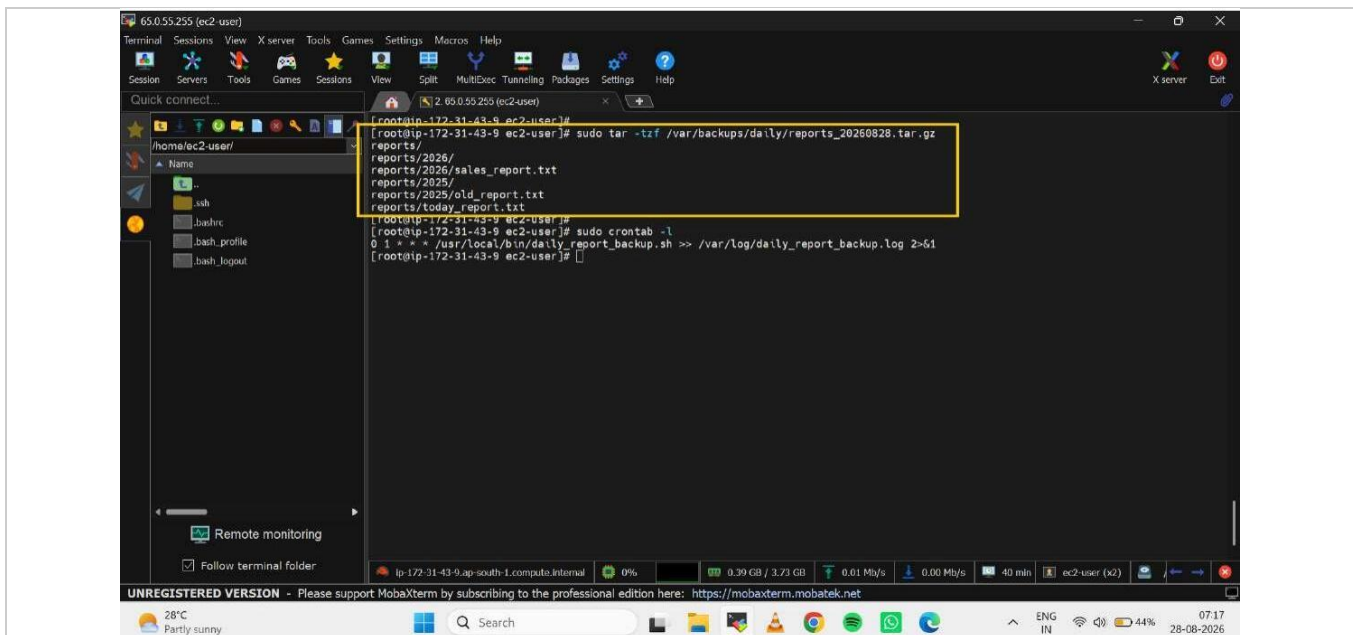


Fig 5: Contents of the new archive verified with `tar -tzf`, and the scheduled crontab job.

8. Summary of Results

- `daily_report_backup.sh` created at `/usr/local/bin` and made executable (`chmod +x`)
- Script creates a timestamped `.tar.gz` of `/var/app/reports` in `/var/backups/daily`
- Archives older than 3 days are moved from `/daily` to `/longterm` using `find -mtime +3`
- Archives older than 7 days are deleted from `/longterm` using `find -mtime +7`
- `crond` service enabled and running; job scheduled via `crontab -e` for 01:00 AM daily
- Manual run verified: new archive created correctly and old archives handled as expected
- All actions logged to `/var/log/daily_report_backup.log` for future auditing